

Ćwiczenie laboratoryjne nr.6

Ksawery Zelek

Fragmenty kodów

Czyszczenie i przygotowanie środowiska

```
4 clear all
5 clc
```

Na początek czyścimy konsolę i pamięć, żeby nie mieszały się nam stare wyniki z poprzednich zadań.

METODA 1 – Metoda rozkładu LU

Skrypt dokonuje dekompozycji macierzy współczynników **A** na macierz trójkątną dolną **L** oraz macierz trójkątną górną **U**. Proces odbywa się na macierzy rozszerzonej o wektor wyrazów wolnych **B**. Po wyznaczeniu macierzy trójkątnych, przeprowadzane jest podstawianie w tył, aby uzyskać wektor rozwiązań **x**.

SKRYPT:

```
8 b = [ 11; 12; 13; 14 ];
9 A = [ 1, 2, 3, 4; 2, 3, 4, 1; 3, 4, 1, 2; 4, 1, 2, 3 ];
10
11 [nb, mb] = size(b);
12 [n, m] = size(A);
13 L = eye(n);
14 U = [ A, b ];
15
16 for k = 1 : m - 1
17     L(k+1:n, k) = U(k+1:n, k) / U(k, k);
18     U(k+1:n, :) = U(k+1:n, :) - L(k+1:n, k) * U(k, :);
19 end
20
21 L2 = eye(n);
22 for k = m : -1 : 2
23     L2(1:k-1, k) = U(1:k-1, k) / U(k, k);
24     U(1:k-1, :) = U(1:k-1, :) - L2(1:k-1, k) * U(k, :);
25 end
26
27 U1 = U(:, 1:m);
28 B = U(:, m+1 : m+mb);
29
30 for k = 1 : n
31     x(k, :) = B(k, :) / U1(k, k);
32 end
33
34 disp('Wynik X (Metoda LU):'); disp(x);
35 disp('Macierz L:'); disp(L);
36 disp('Macierz L2:'); disp(L2);
37 disp('Macierz U:'); disp(U);
38 disp('Macierz U1:'); disp(U1);
```

WYNIK:

Wynik X (Metoda LU):

2
1
1
1

Macierz L:

1	0	0	0
2	1	0	0
3	2	1	0
4	7	-1	1

Macierz L2:

1.00000000000000e+00	-2.00000000000000e+00	-7.50000000000000e-01	1.00000000000000e-01
0	1.00000000000000e+00	5.00000000000000e-01	-1.75000000000000e-01
0	0	1.00000000000000e+00	1.00000000000000e-01
0	0	0	1.00000000000000e+00

Macierz U:

1	0	0	0	2
0	-1	0	0	-1
0	0	-4	0	-4
0	0	0	40	40

Macierz U1:

1	0	0	0
0	-1	0	0
0	0	-4	0
0	0	0	40

METODA 2 – Metoda Cramera

Skrypt wykorzystuje własności wyznaczników macierzy do rozwiązania układu. Definiowany jest układ równań poprzez wektor **b** i macierz współczynników **A**. Obliczany jest wyznacznik główny **W** za pomocą funkcji wbudowanej **det()**. Wyznaczniki szczegółowe **W_x**, **W_y**, **W_z**, **W_w** są zapisane statycznie dla podanej macierzy **A** poprzez ręczne wstawienie wektora **b** do kolejnych kolumn. Następnie wynik **x** otrzymywany jest przez podzielenie wyznaczników szczegółowych przez główny.

SKRYPT:

```
7      b = [ 1; 2; -3; -6 ];
8      A = [ 2, -1, 1, -1; 2, -1, 0, -3; 3, 0, -1, 1; 2, 2, -2, 5 ];
9
10     W = det(A);
11     Wx = det([ 1, -1, 1, -1; 2, -1, 0, -3; -3, 0, -1, 1; -6, 2, -2, 5 ]);
12     Wy = det([ 2, 1, 1, -1; 2, 2, -3, 0; 3, -3, -1, 1; 2, -6, -2, 5 ]);
13     Wz = det([ 2, -1, 1, -1; 2, -1, 2, 0; 3, 0, -3, 1; 2, 2, -6, 5 ]);
14     Ww = det([ 2, -1, 1, 1; 2, -1, -3, 2; 3, 0, -1, -3; 2, 2, -2, -6 ]);
15
16     x = [ Wx/W; Wy/W; Wz/W; Ww/W ];
17
18     disp('Wyznacznik główny W:'); disp(W);
19     disp('Wyniki (x1, x2, x3, x4) (Cramer):'); disp(x);
```

WYNIK:

```
Wyznacznik główny W:
-8.999999999999996e+00

Wyniki (x1, x2, x3, x4) (Cramer):
-1.480297366166876e-16
1.100000000000000e+01
-3.333333333333335e-01
-3.333333333333334e+00
```

METODA 3 – Metoda Gaussa

Skrypt rozwiązuje układ równań poprzez sprowadzenie układu do macierzy trójkątnej górnej, co umożliwia podstawienie w górę (wyznaczenie wektora $\mathbf{X}$). Kod zawiera funkcję częściowego wyboru elementu głównego – w przypadku natrafienia na wartość 0 na przekątnej $\mathbf{A}(j,j) == 0$, program szuka wiersza poniżej, w którym dany współczynnik jest różny od zera i zamienia je miejscami (pętla z warunkiem $\text{if } \mathbf{A}(k,j) \neq 0$).

SKRYPT:

```
6      b = [ 1; 2; -3; -6 ];
7      A = [ 2, -1, 1, -1; 2, -1, 0, -3; 3, 0, -1, 1; 2, 2, -2, 5 ];
8
9      r = b;
10     s = 0;
11     [n, m] = size(A);
12
13     for j = 1 : n - 1
14         if A(j, j) == 0
15             % POPRAWIONA ZAMIANA WIERSZY
16             for k = j + 1 : n
17                 if A(k, j) ~= 0
18                     % Zapisz stary wiersz J do pamięci
19                     temp_A = A(j, :);
20                     temp_r = r(j);
21
22                     % Wstaw wiersz K w miejsce wiersza J
23                     A(j, :) = A(k, :);
24                     r(j) = r(k);
25
26                     % Wstaw zapamiętany stary wiersz J w miejsce K
27                     A(k, :) = temp_A;
28                     r(k) = temp_r;
29                     break;
30             end
31         end
32     end
```

```

34     for i = 1 + s : n - 1
35         L = A(i+1, j) / A(j, j);
36         A(i+1, :) = A(i+1, :) - L * A(j, :);
37         r(i+1) = r(i+1) - L * r(j);
38     end
39     s = s + 1;
40 end
41
42 x(n) = r(n) / A(n, n);
43 for i = n - 1 : -1 : 1
44     sum_val = 0;
45     for j = i + 1 : n
46         sum_val = sum_val + A(i, j) * x(j);
47     end
48     x(i) = (1 / A(i, i)) * (r(i) - sum_val);
49 end
50
51 disp('Wynik metody Gaussa:');
52 disp(x');

```

WYNIK:

```

Wynik metody Gaussa:
-2.220446049250313e-16
 1.999999999999999e+00
 1.666666666666667e+00
-1.333333333333333e+00

```

METODA 4 – Metody wbudowane

Zastosowanie dwóch komend systemowych: operatora dzielenia lewostronnego macierzy $A \setminus B$ oraz komendy wykorzystującej mnożenie przez odwróconą macierz $x = \text{inv}(A) * b$.

SKRYPT:

```
6      b = [ 1; 2; -3; -6 ];
7      A = [ 2, -1, 1, -1; 2, -1, 0, -3; 3, 0, -1, 1; 2, 2, -2, 5 ];
8
9      % NASZ PROGRAM dla metody dzielenia lewostronnego A\b
10     disp('Wynik A\b:');
11     x = A\b;
12     disp(x);
13
14     % NASZ PROGRAM dla metody inv(A)*b
15     disp('Wynik inv(A)*b:');
16     x = inv(A)*b;
17     disp(x);
```

WYNIK:

```
Wynik A\b:
    7.401486830834377e-17
    2.000000000000000e+00
    1.666666666666667e+00
   -1.333333333333333e+00

Wynik inv(A)*b:
                                0
    2.000000000000000e+00
    1.666666666666667e+00
   -1.333333333333333e+00
```